

PORTADA

Técnicas de Programación

Unidad 3 — Técnicas de Programación: Documentación y Optimización de Código

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Un código sin documentar es una pregunta que solo su autor puede responder — y a veces ni siquiera él, seis meses después.



TRANSICIÓN

Cerrando la Unidad 3

Ya viste cómo estructurar código con SOLID, cómo escribirlo de forma limpia, y cómo compartirlo en equipo con GitHub. Cierra la unidad con dos frentes complementarios: **documentar** el código para quien lo lea después, y **optimizarlo** con criterio, sin sacrificar todo lo anterior por rendimiento que a veces ni siquiera hace falta.

CONCEPTO

Niveles de documentación

Documentar no es solo escribir comentarios — hay distintos niveles, cada uno con una audiencia distinta:

Nivel	Para quién	Ejemplo
Inline (comentarios)	Quien lee ese método específico	<code>// La API exige reintentar 3 veces</code>
Javadoc	Quien va a <i>usar</i> una clase/método sin leer su implementación	<code>/** Calcula el total... */</code>
README	Quien llega nuevo al proyecto	Qué hace, cómo instalarlo, cómo ejecutarlo

CONCEPTO

Javadoc: documentación que se puede generar

Javadoc es la convención estándar de Java para documentar clases y métodos con comentarios especiales (`/** ... */`) que herramientas como `javadoc` pueden convertir en páginas HTML de referencia — muchas librerías de Java que ya has usado (por ejemplo, `ArrayList` o `Scanner`) están documentadas así.

```
/**
 * Calcula el total de una factura aplicando el IVA vigente.
 *
 * @param subtotal monto antes de impuestos, debe ser mayor a 0
 * @return el total con IVA incluido
 */
public double calcularTotal(double subtotal) {
    return subtotal * 1.19;
}
```

CONTRAEJEMPLO

Documentar lo obvio vs. documentar el porqué

```
/** Este método suma dos números. @param a, b los números @return la suma */  
public int sumar(int a, int b) { return a + b; }
```

Ese Javadoc no aporta nada que el código no diga ya. Un Javadoc útil documenta lo que **no es obvio** con solo leer la firma: reglas de negocio, casos límite, o por qué el método existe así.

```
/** Redondea al peso más cercano según la política contable vigente (NIIF 15, 4.2). */  
public long redondearComision(double comision) { ... }
```

CONCEPTO

README: la puerta de entrada al proyecto

Un buen README responde, en orden, las primeras preguntas de cualquiera que abre el repositorio:

- **Qué hace** el proyecto (una o dos frases).
- **Cómo instalarlo** (dependencias, versión de Java requerida).
- **Cómo ejecutarlo** (comando exacto, o cómo correrlo desde el IDE).
- **Cómo contribuir** (si es un proyecto de equipo: convención de ramas, cómo abrir un PR — lo visto la sesión pasada).

TRANSICIÓN

De documentar a optimizar

Documentar ayuda a que el código se entienda. La **optimización** es el otro lado de la moneda: hacer que el código use menos tiempo o memoria de la que necesitaba antes — pero, igual que con el código limpio, hay una forma correcta y una forma que sale cara de hacerlo.

CONCEPTO

¿Qué es optimizar? Midiendo con Big-O

Optimizar es reducir el tiempo o los recursos que un programa necesita para resolver un problema. La notación **Big-O** describe cómo crece el tiempo de ejecución de un algoritmo a medida que crecen los datos de entrada (n) — no mide segundos exactos, mide la **tendencia de crecimiento**.

CONCEPTO

Complejidades comunes

Notación	Se lee	Ejemplo en Java
$O(1)$	Constante	Acceder a <code>arreglo[i]</code>
$O(\log n)$	Logarítmica	Búsqueda binaria en un arreglo ordenado
$O(n)$	Lineal	Un <code>for</code> que recorre una lista una vez
$O(n^2)$	Cuadrática	Dos <code>for</code> anidados sobre la misma lista

A medida que n crece, un algoritmo $O(n^2)$ se vuelve dramáticamente más lento que uno $O(n)$ — la diferencia se nota poco con 10 elementos, y mucho con 100.000.

TENSIÓN CONCEPTUAL

"La optimización prematura es la raíz de todos los males"

"We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil." — Donald Knuth, *Structured Programming with go to Statements* (1974)

Knuth no dice que optimizar esté mal — dice que hacerlo **antes de saber dónde está el problema real** desperdicia tiempo y, casi siempre, sacrifica la legibilidad del código (todo lo trabajado en la Unidad 3) a cambio de una mejora que ni siquiera se necesitaba.

CONCEPTO

Cuándo optimizar: mide, no adivines

El criterio correcto no es "esto se ve lento", es **medir**: identificar con datos reales (tiempo de ejecución, número de llamadas) qué parte del programa consume más recursos, antes de tocarla. Herramientas de *profiling* del IDE muestran exactamente eso. Optimizar una función que se ejecuta una vez al iniciar el programa, cuando el problema real está en un método que se llama miles de veces, es esfuerzo desperdiciado.

CONCEPTO

Un caso concreto: buscar en una lista vs. un mapa

```
// 0(n): revisa la lista completa en el peor caso
ArrayList<Estudiante> lista = new ArrayList<>();
Estudiante e = lista.stream()
    .filter(est -> est.getId().equals(id))
    .findFirst().orElse(null);

// 0(1): acceso directo por clave
HashMap<String, Estudiante> mapa = new HashMap<>();
Estudiante e = mapa.get(id);
```

Si el programa busca estudiantes por id constantemente, cambiar la estructura de datos (de `ArrayList` a `HashMap`) es una optimización real y medible — no una micro-optimización especulativa.

CASO

Un guion faltante y un cohete perdido

En julio de 1962, el cohete **Mariner 1** de la NASA, destinado a sobrevolar Venus, tuvo que ser destruido apenas 293 segundos después del despegue porque se desvió de su trayectoria. La investigación encontró que el software de guiado tenía un error: al transcribir una fórmula matemática de las notas escritas a mano al código, faltaba una **barra superior** sobre una variable (que indicaba un valor promediado) — un detalle mínimo, mal documentado en las especificaciones originales, que el código terminó implementando de forma incorrecta.

El incidente, ampliamente documentado en la historia de la ingeniería de software, se cita a menudo como recordatorio de que la **precisión en los detalles y su documentación** —no solo la velocidad de ejecución— es parte de escribir código correcto.

INTERACCIÓN

Identifiquen la complejidad y optimicen

En parejas: identifiquen la complejidad Big-O de este fragmento y propongan cómo reducirla.

```
public boolean tieneDuplicados(ArrayList<Integer> numeros) {  
    for (int i = 0; i < numeros.size(); i++) {  
        for (int j = 0; j < numeros.size(); j++) {  
            if (i != j && numeros.get(i).equals(numeros.get(j))) {  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

7 minutos · pista: piensen en un HashSet

SÍNTESIS — CIERRE DE LA UNIDAD 3

Lo que hay que llevarse de la unidad

1. Javadoc y README documentan para audiencias distintas — quien usa el código, y quien llega nuevo al proyecto.
2. Documenta el porqué, no lo obvio; Big-O mide cómo crece el tiempo de ejecución con el tamaño de los datos.
3. Mide antes de optimizar — la optimización prematura sacrifica legibilidad por una ganancia que a veces no existe.
4. **Resumen de toda la Unidad 3:** SOLID (cómo dividir responsabilidades), código limpio (cómo escribir cada método), GitHub (cómo colaborar sin perder el control) y documentación/optimización (cómo dejar el código listo para otros y para que crezca).



CIERRE

Antes de la próxima unidad

- Repasar las cuatro sesiones de la Unidad 3 — el **Parcial Final de Técnicas de Programación** evalúa estos temas (SOLID, código limpio, GitHub, documentación y optimización), y se aplica en la semana del 30 de noviembre al 5 de diciembre.
- Aplicar al menos una práctica de documentación u optimización a un proyecto propio antes de la siguiente unidad.

Próxima unidad: Archivos, Errores/Excepciones y Pruebas.

